UNIVERSITATEA DE STAT DIN MOLDOVA
FACULTATEA DE FIZICĂ ȘI INGINERIE
DEPARTAMENTUL FIZICA APLICATĂ ȘI INFORMATICĂ

BUIMESTRU SILVIA

APLICAȚIE EDUCAȚIONALĂ DE TIP JOC PENTRU ACTIVITĂȚI DE GRUP

0613.1 TEHNOLOGIA INFORMAȚIEI

Teza de licență

Şef Departament: _____
(semnătura)

Conducător ştiinţific: _____ **Sprincean Veaceslav, conferenţiar universitar**
(semnătura)

Autorul: _____
(semnătura)

CHIȘINĂU-2026

Cuprins

Cuprins

Lista abrevierilor

Introducere

1. Fundamente teoretice, context educațional și relevanța temei

1.1. Digitalizarea procesului educațional
1.2. Jocul educațional și mecanismele de gamificare
1.3. Învățarea colaborativă și competiția controlată
1.4. Modelul vizual inspirat de fizică
1.5. Relevanța tehnologiilor web în instrumentele educaționale
2. Analiza domeniului și specificarea cerințelor

2.1. Problema abordată și contextul de utilizare
2.2. Utilizatori și scenarii pedagogice
2.3. Scopul lucrării și obiectivele operaționale
2.4. Cerințe funcționale
2.5. Cerințe nefuncționale și constrângeri
2.6. Riscuri, ipoteze și limite inițiale
3. Proiectarea aplicației

3.1. Arhitectura generală și structura proiectului
3.2. Modelul de date și banca de întrebări
3.3. Fluxul de interacțiune utilizator-sistem
3.4. Modelarea stării prin obiectul gameState
3.5. Regulile jocului și mecanismele de feedback
3.6. Extensibilitate și posibilități de evoluție
4. Implementarea interfeței cu HTML și CSS

4.1. Structura HTML a interfeței
4.2. Organizarea vizuală prin CSS
4.3. Panourile de interacțiune pentru echipe
4.4. Zona centrală și componenta multimedia
4.5. Elemente de responsive design și accesibilitate
4.6. Evaluarea interfeței și direcții de rafinare
5. Implementarea logicii aplicației în JavaScript

5.1. Încărcarea întrebărilor și utilizarea funcției fetch
5.2. Selectarea întrebărilor și evitarea repetării
5.3. Gestionarea intrării numerice și validarea răspunsurilor
5.4. Actualizarea scorului și feedbackul vizual
5.5. Ciclul de viață al jocului: start, finalizare, resetare
5.6. Mentenabilitate, claritate și oportunități de refactorizare
6. Testare, evaluare și perspective de dezvoltare
6.1. Metodologia de testare
6.2. Scenarii funcționale de verificare
6.3. Valoarea pedagogică a aplicației
6.4. Limitări tehnice și funcționale
6.5. Direcții de dezvoltare ulterioară
6.6. Recomandări pentru exploatarea practică
Concluzii și recomandări
Bibliografie
Anexe

Lista abrevierilor

Abreviere	Semnificație
HTML	HyperText Markup Language
CSS	Cascading Style Sheets
JS	JavaScript
DOM	Document Object Model
JSON	JavaScript Object Notation
HTTP	Hypertext Transfer Protocol
API	Application Programming Interface
UI	User Interface
UX	User Experience
SPA	Single Page Application
MP4	MPEG-4 Part 14
TOC	Table of Contents

Introducere

Transformarea digitală a educației a determinat apariția unor instrumente software orientate nu doar spre prezentarea informației, ci și spre implicarea activă a elevului în procesul de învățare. În special în cazul disciplinelor care cer exercițiu repetat, precum matematica elementară, mediile interactive pot susține atenția, pot reduce monotonia și pot oferi feedback imediat. În această lucrare este analizată și documentată realizarea unei aplicații educaționale de tip joc, construită pentru activități de grup și bazată pe o competiție vizuală între două echipe.

Tema lucrării pornește de la o problemă pedagogică concretă: elevii se implică mai greu în exercițiile de calcul atunci când acestea sunt prezentate exclusiv în formă statică, fără componentă ludică și fără un răspuns vizibil imediat. Proiectul propus transformă rezolvarea unor exerciții simple de matematică într-o activitate de echipă, susținută de o interfață grafică expresivă. Fiecare răspuns corect produce un efect vizual observabil, astfel încât relația dintre acțiune și rezultat devine clară, motivantă și ușor de urmărit.

Aplicația este implementată folosind tehnologii web accesibile și stabile: HTML pentru structura paginii, CSS pentru design și organizarea interfeței, JavaScript pentru logica jocului, iar un fișier JSON pentru banca de întrebări. În forma actuală, proiectul conține 10 întrebări de aritmetică elementară. Această alegere tehnologică permite utilizarea aplicației pe calculatoare obișnuite, pe tablă interactivă sau în laborator, fără infrastructură software complexă.

Lucrarea are atât o componentă tehnică, cât și una metodologică. Din punct de vedere tehnic sunt analizate structura fișierelor, arhitectura interfeței, modelul de stare al aplicației, gestionarea întrebărilor și mecanismele de feedback vizual. Din punct de vedere educațional sunt discutate rolul competiției controlate, colaborarea în echipă, relevanța analogiilor vizuale și modul în care un joc simplu poate susține exersarea rapidă și repetată.

Metodele utilizate în lucrare includ analiza surselor teoretice privind gamificarea și învățarea activă, analiza codului sursă al aplicației, descrierea arhitecturală a soluției, definirea cerințelor funcționale și nefuncționale, precum și formularea unor scenarii de testare și exploatare practică. În acest sens, documentul nu este doar o prezentare narativă a rezultatului final, ci și o documentație tehnică menită să explice de ce proiectul funcționează, cum poate fi extins și ce limite prezintă în varianta curentă.

Structura tezei este organizată astfel: primul capitol fundamentează tema din perspectivă teoretică și pedagogică; al doilea capitol formulează cerințele și contextul de utilizare; al treilea

capitol descrie proiectarea generală a aplicației; capitolele patru și cinci analizează implementarea interfeței și a logicii JavaScript; capitolul șase prezintă testarea, evaluarea și direcțiile de dezvoltare; în final sunt formulate concluziile, bibliografia și anexele tehnice, inclusiv listingurile sursă relevante.

1. Fundamente teoretice, context educațional și relevanța temei

1.1. Digitalizarea procesului educațional

Educația contemporană este marcată de trecerea de la modele de predare centrate exclusiv pe expunerea frontală la modele în care elevul interacționează constant cu resurse digitale, aplicații și medii vizuale. Acest proces nu înseamnă doar înlocuirea suportului pe hârtie cu un ecran, ci și apariția unor noi forme de reprezentare a conținutului și a unor noi modalități de exersare. Platformele interactive, simulările și jocurile educaționale oferă profesorului posibilitatea de a organiza activități în care elevul primește imediat confirmarea corectitudinii sau incorectitudinii propriilor acțiuni.

În cazul matematicii, avantajul digitalizării constă în faptul că abstractul poate fi convertit în acțiune și imagine. Un calcul reușit nu mai rămâne doar un rezultat numeric, ci poate produce o schimbare într-o scenă, într-un scor, într-o animație sau într-un obiect grafic. Astfel, elevul nu percepe exercițiul ca pe o sarcină izolată, ci ca pe o componentă a unei experiențe dinamice. Din această perspectivă, aplicațiile simple, dezvoltate cu tehnologii web, pot avea un rol semnificativ în completarea activităților tradiționale.

1.2. Jocul educațional și mecanismele de gamificare

Conceptul de gamificare presupune introducerea unor mecanisme specifice jocurilor în contexte non-ludice, cu scopul de a crește motivația și implicarea participanților. Printre aceste mecanisme se numără scorul, progresul vizibil, feedbackul imediat, provocarea gradată și competiția controlată. Într-un context educațional, ele nu trebuie aplicate în mod superficial, ci integrate astfel încât să susțină obiectivele didactice și să nu transforme activitatea într-o simplă cursă după puncte.

Aplicația analizată în această lucrare folosește o formă de gamificare minimalistă, dar eficientă. Există două echipe, fiecare primește o întrebare, introduce răspunsul printr-o interfață de tip calculator și observă cum scorul influențează scena centrală. Mecanica este simplă, iar tocmai această simplitate o face potrivită pentru utilizarea în clasă. Elevii pot înțelege repede regulile, iar profesorul poate folosi timpul disponibil pentru exercițiu și discuție, nu pentru explicarea unui set complicat de operații.

1.3. Învățarea colaborativă și competiția controlată

Activitățile desfășurate pe echipe stimulează nu doar rezolvarea individuală, ci și negocierea rapidă a unei soluții, distribuirea rolurilor și comunicarea dintre participanți. Într-un joc educațional

bine calibrat, elevii cooperează în interiorul echipei și concurează în același timp cu cealaltă echipă. Această dublă dinamică poate crește participarea chiar și în cazul elevilor mai rezervați, deoarece presiunea performanței este împărțită între mai mulți colegi, iar succesul este perceput ca rezultat colectiv.

Competiția devine pedagogic utilă atunci când regulile sunt transparente, feedbackul este imediat, iar diferențele de punctaj pot fi recuperate. Aplicația propusă respectă aceste condiții prin afișarea simultană a scorurilor, prin blocarea interacțiunii înainte de începerea jocului și prin limitarea rundei la atingerea unui prag clar de victorie. Astfel se evită ambiguitatea și se creează un cadru de lucru predictibil, ușor de condus de către profesor.

1.4. Modelul vizual inspirat de fizică

Un element distinctiv al proiectului este folosirea unei scene de tip tragere de funie ca metaforă vizuală a progresului. Din punct de vedere didactic, această alegorie are două avantaje. În primul rând, traduce competiția într-o imagine ușor de înțeles: dacă o echipă rezolvă mai multe exerciții corect, aceasta „trage” jocul în favoarea ei. În al doilea rând, sugerează ideea de echilibru și dezechilibru, noțiuni care au legături intuitive cu fizica și cu reprezentarea forțelor.

Este important de precizat că aplicația nu încearcă să simuleze riguros un fenomen fizic. Scopul ei nu este calculul forțelor, ci utilizarea unei reprezentări intuitive pentru a sprijini înțelegerea relației dintre performanță și rezultat. Prin urmare, analogia cu fizica este una conceptuală, nu experimentală. Această abordare este potrivită pentru lucrare deoarece demonstrează că modele vizuale simple pot media trecerea de la exercițiul abstract la o experiență interactivă, păstrând claritatea regulilor.

1.5. Relevanța tehnologiilor web în instrumentele educaționale

Tehnologiile web sunt deosebit de potrivite pentru aplicațiile educaționale cu distribuție rapidă. Un browser modern este suficient pentru rularea unui astfel de produs, ceea ce reduce barierele de instalare și simplifică adoptarea în școală. HTML structurează conținutul, CSS controlează aspectul vizual, iar JavaScript oferă interactivitate și actualizare în timp real. Pentru proiecte mici și medii, această combinație este suficientă pentru a crea experiențe robuste și ușor de întreținut.

În plus, tehnologiile web facilitează extensia ulterioară a aplicației. Un proiect pornit ca soluție locală, fără backend, poate fi extins ulterior cu autentificare, panou pentru profesor, salvarea progresului sau generarea dinamică a întrebărilor. Alegerea unei baze front-end clare în faza inițială

permite evoluția graduală a produsului. În cazul de față, aplicația este intenționat simplă și concentrată pe scenariul de utilizare din clasă, însă arhitectura ei permite dezvoltări viitoare fără rescriere completă.

2. Analiza domeniului și specificarea cerințelor

2.1. Problema abordată și contextul de utilizare

Problema principală pe care o abordează proiectul este scăderea atenției și a motivației în activitățile repetitive de calcul. În practică, exercițiile de adunare și scădere sunt esențiale, dar prezentate exclusiv în format tradițional pot deveni previzibile și lipsite de atractivitate. Aplicația urmărește să păstreze același conținut de bază, dar să îl livreze într-o formă care produce reacție imediată și implicare de grup.

Contextul de utilizare vizat este sala de clasă, în special situațiile în care profesorul are nevoie de un instrument rapid pentru activarea elevilor la începutul lecției, pentru consolidarea cunoștințelor sau pentru încheierea unei activități de recapitulare. Interfața mare și butoanele evidente recomandă produsul pentru proiecție pe ecran sau pentru tablă interactivă. Totodată, structura simplă permite rularea și pe un calculator obișnuit, fără conturi, baze de date sau configurări complicate.

2.2. Utilizatori și scenarii pedagogice

În forma actuală, pot fi identificați trei actori principali: profesorul, elevii organizați în două echipe și operatorul aplicației. În multe situații profesorul poate avea și rolul de operator, însă în clasele mari este posibil ca un elev sau un asistent să introducă efectiv răspunsurile. Această separare este utilă deoarece permite învățătorului să se concentreze pe coordonarea didactică și pe observarea comportamentului echipelor, nu doar pe controlul interfeței.

Scenariul de bază este următorul: profesorul pornește jocul, echipele primesc câte o întrebare, discută rapid și introduc rezultatul folosind calculatorul numeric de pe ecran. Dacă răspunsul este corect, scorul se actualizează și scena centrală se deplasează în favoarea echipei; dacă răspunsul este greșit, se afișează un feedback vizual de eroare. Activitatea continuă până când una dintre echipe atinge scorul final. Acest flux este suficient de simplu pentru a fi repetat de mai multe ori în aceeași lecție.

2.3. Scopul lucrării și obiectivele operaționale

Scopul lucrării este proiectarea și documentarea unei aplicații educaționale interactive care să transforme rezolvarea exercițiilor matematice într-o activitate de grup atractivă, vizibilă și ușor de administrat. Din perspectiva produsului software, scopul este obținerea unei aplicații clare, stabile și

suficient de flexibile pentru extindere. Din perspectivă educațională, scopul este stimularea participării, colaborării și reacției rapide.

Pentru atingerea acestui scop au fost formulate mai multe obiective operaționale: definirea structurii interfeței, implementarea unui model de stare pentru joc, încărcarea întrebărilor dintr-o sursă externă, verificarea răspunsurilor, afișarea scorului, reprezentarea vizuală a progresului, precum și documentarea limitărilor și a direcțiilor de dezvoltare. Aceste obiective asigură faptul că lucrarea tratează atât produsul final, cât și procesul tehnic prin care acesta a fost obținut.

2.4. Cerințe funcționale

Cerințele funcționale definesc comportamentul minim necesar pentru ca aplicația să poată fi utilizată în activități didactice reale. Ele descriu operațiile pe care utilizatorul le poate executa, reacțiile sistemului și condițiile de finalizare a jocului. Într-un proiect de dimensiuni reduse, formularea explicită a acestor cerințe este importantă deoarece previne extinderea necontrolată a funcționalității și permite evaluarea obiectivă a rezultatului final.

Pentru această aplicație, cerințele funcționale sunt strâns legate de fluxul jocului: pornirea, încărcarea întrebărilor, introducerea răspunsurilor, validarea acestora, actualizarea scorurilor și anunțarea câștigătorului. În tabelul următor sunt sintetizate cerințele esențiale și modul în care ele sunt reflectate în produsul software.

Cod	Cerință	Reflecție în aplicație
CF1	Pornirea jocului prin buton dedicat	Implementată prin funcția startGame() și butonul START GAME
CF2	Încărcarea întrebărilor din sursă externă	Realizată prin fetch('questions.json')
CF3	Atribuirea unei întrebări pentru fiecare echipă	Gestionată prin loadNewQuestion(team)
CF4	Introducerea numerică a răspunsurilor	Realizată prin calculatorul virtual și display-ul asociat
CF5	Validarea răspunsului introdus	Realizată în submitAnswer(team)
CF6	Actualizarea scorului și afișarea lui	Realizată prin gameState și updateScore(team)
CF7	Feedback vizual pozitiv/negativ	Schimbarea temporară a culorii întrebării și deplasarea scenei
CF8	Finalizarea jocului la pragul de 10 puncte	Verificată în submitAnswer(team) și tratată de endGame()
CF9	Resetarea completă a jocului	Realizată prin resetGame()
CF10	Blocarea interacțiunii înainte de start și după final	Gestionată prin clasa CSS blocked și funcția setTeamColumnsBlocked()

2.5. Cerințe nefuncționale și constrângeri

Cerințele nefuncționale descriu calitățile pe care aplicația trebuie să le aibă dincolo de simpla existență a funcțiilor sale. În context educațional, acestea sunt foarte importante deoarece un instrument poate fi corect din punct de vedere logic, dar greu de folosit, slab lizibil sau dificil de lansat în clasă. Pentru un joc destinat unor activități scurte, simplitatea, claritatea și stabilitatea devin criterii esențiale.

În proiectul de față, accentul este pus pe accesibilitatea lansării, lizibilitatea elementelor, reacția rapidă la interacțiuni și posibilitatea de întreținere a codului. De asemenea, trebuie remarcat faptul că aplicația rulează în întregime pe partea client, ceea ce simplifică arhitectura, dar introduce și limite, cum ar fi absența stocării persistente sau a unui mecanism centralizat de administrare. Tabelul de mai jos sintetizează aceste aspecte.

Cod	Criteriu	Argument
CNF1	Ușurință în lansare	Aplicația rulează local din browser, fără backend și fără baze de date
CNF2	Lizibilitate	Butoane mari, scor vizibil, structură în trei zone distincte
CNF3	Timp redus de răspuns	Actualizările de interfață sunt locale și imediate
CNF4	Mentenabilitate	Separarea structurii, stilului, logicii și conținutului
CNF5	Portabilitate	Funcționează în browsere moderne pe calculator și tabletă
CNF6	Extensibilitate	Banca de întrebări poate fi extinsă fără schimbări arhitecturale majore
CNF7	Claritate pedagogică	Reguli puține, feedback imediat, progres vizual explicit
CNF8	Fiabilitate minimală	Starea jocului este centralizată în gameState și poate fi resetată complet

2.6. Riscuri, ipoteze și limite inițiale

Orice proiect software, chiar și unul de dimensiuni reduse, trebuie analizat și prin prisma riscurilor. În cazul acestei aplicații, principalul risc ține de conținut: banca actuală include numai 10 întrebări, ceea ce este suficient pentru demonstrație și jocuri scurte, dar poate deveni repetitiv în utilizarea frecventă. O altă limită este faptul că răspunsurile sunt comparate sub formă de șiruri de caractere, ceea ce face necesară introducerea exactă a rezultatului așteptat.

Au fost asumate câteva ipoteze de lucru: utilizatorii dispun de un browser modern, fișierul JSON cu întrebări este disponibil în aceeași locație cu pagina, iar jocul este folosit într-un cadru supravegheat de profesor. În plus, s-a considerat că nivelul de dificultate al exercițiilor este adecvat

pentru activități de încălzire și consolidare. Documentarea acestor limite este utilă pentru că explică de ce soluția actuală este simplă și ce zone trebuie tratate prioritar într-o versiune ulterioară.

3. Proiectarea aplicației

3.1. Arhitectura generală și structura proiectului

Aplicația are o arhitectură front-end compactă, bazată pe separarea clară a responsabilităților între fișiere. Structura este formată dintr-un fișier HTML care definește interfața, un fișier CSS care stabilește designul și poziționarea componentelor, un fișier JavaScript care gestionează logica jocului, un fișier JSON care conține setul de întrebări și un director de resurse multimedia. Această organizare este potrivită pentru un produs demonstrativ și educațional deoarece permite identificarea rapidă a fiecărei părți a sistemului.

Un avantaj important al acestei arhitecturi este că poate fi înțeleasă ușor și de către studenți sau elevi care învață bazele dezvoltării web. Fiecare fișier are un rol bine delimitat, iar fluxul de date este simplu: HTML expune elementele ce vor fi manipulate, CSS le stilizează, JavaScript citește întrebările și actualizează interfața, iar resursele media susțin feedbackul vizual. În tabelul următor sunt sintetizate componentele de bază ale proiectului.

Fișier / director	Rol
index.html	Definește structura completă a interfeței, zonele vizuale și ancorele DOM pentru actualizare din JavaScript
styles.css	Controlează layout-ul pe trei coloane, stilurile butoanelor, culorile echipelor și efectele vizuale
game.js	Conține logica jocului, gestionarea stării, încărcarea întrebărilor și reacțiile de interfață
questions.json	Stochează banca de întrebări și răspunsuri într-un format simplu și extensibil
assets/	Include imaginea tematică și videoclipurile folosite pentru feedback vizual
package.json	Păstrează scriptul de pornire și dependența folosită pentru servirea locală a aplicației

3.2. Modelul de date și banca de întrebări

Datele aplicației sunt stocate într-un format minimal: fiecare întrebare este reprezentată printr-un obiect JSON cu două proprietăți, `question` și `answer`. Această alegere reduce complexitatea și permite extinderea ușoară a conținutului fără modificări în logica principală a jocului. Fișierul conține în prezent 10 itemi, orientați spre adunări și scăderi scurte. Exemple reprezentative din bancă sunt: What is 25 + 25?; What is 85 + 13?; What is 42 - 3?; What is 33 - 12?.

Modelul de date este suficient pentru varianta curentă, însă documentarea lui arată și direcțiile posibile de dezvoltare. Într-o versiune viitoare, fiecare întrebare ar putea include un nivel de dificultate, o categorie curriculară, un timp recomandat de rezolvare sau o explicație pentru

răspuns. Chiar și fără aceste extensii, soluția actuală demonstrează un principiu important: separarea conținutului de logică simplifică întreținerea și permite reutilizarea aceleiași aplicații în mai multe contexte educaționale.

3.3. Fluxul de interacțiune utilizator-sistem

Fluxul principal de interacțiune este liniar și intenționat simplificat. La început, coloanele echipelor sunt blocate și afișează mesajul că jocul trebuie pornit. După apăsarea butonului de start, scorurile sunt resetate, întrebările sunt încărcate pentru ambele echipe, iar zona centrală indică faptul că jocul este în desfășurare. Ulterior, elevii introduc răspunsurile prin butoanele numerice, iar sistemul reacționează imediat la fiecare trimitere.

Această organizare sprijină utilizarea în clasă deoarece reduce la minimum numărul de pași și de decizii operaționale. Profesorul nu trebuie să configureze utilizatori, sesiuni sau setări suplimentare. Regula este simplă: pornești jocul, rezolvi întrebările, urmărești scorul, oprești jocul la victorie sau îl resetezi pentru o nouă rundă. Un asemenea flux este adecvat pentru activități scurte și recurente, unde timpul de pregătire trebuie să fie foarte mic.

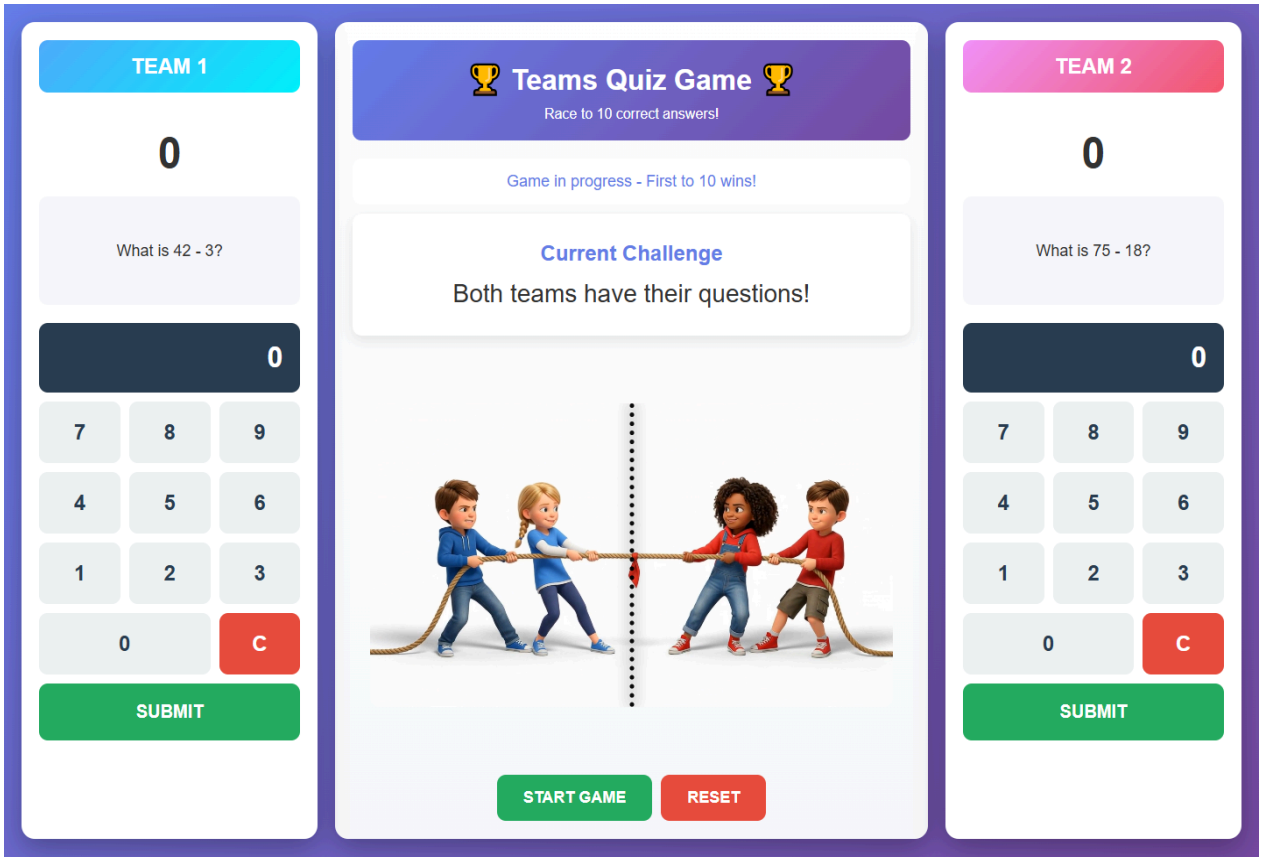

Figura 1. Imagine conceptuală asociată competiției dintre două echipe într-un joc educațional de tip tragere de funie.

3.4. Modelarea stării prin obiectul gameState

Un element central al arhitecturii logice este obiectul `gameState`, care reține dacă jocul este activ, scorurile fiecărei echipe, întrebările curente, seturile de întrebări deja utilizate și conținutul afișajelor numerice. Această decizie de proiectare este importantă deoarece concentrează informațiile volatile într-un singur loc și reduce dependențele implicite dintre funcții. În loc ca fiecare operație să citească și să modifice mai multe surse dispersate, aplicația lucrează cu un model comun de stare.

Din perspectivă didactică, `gameState` este și un exemplu foarte bun pentru explicarea principiului „single source of truth" într-o aplicație interactivă. Chiar dacă proiectul nu folosește un framework modern, ideea de bază rămâne aceeași: comportamentul interfeței trebuie să derive dintr-o stare internă coerentă. O astfel de organizare face codul mai ușor de urmărit și permite, ulterior, mutarea logicii într-o arhitectură modulară sau într-un framework de componentizare.

3.5. Regulile jocului și mecanismele de feedback

Regulile jocului sunt puține, dar clar definite: fiecare echipă primește o întrebare, fiecare răspuns corect crește scorul cu o unitate, iar jocul se încheie când una dintre echipe ajunge la pragul de zece puncte. Feedbackul pozitiv este exprimat prin actualizarea scorului, schimbarea temporară a culorii zonei de întrebare și modificarea poziției scenei centrale. Feedbackul negativ constă în evidențierea răspunsului greșit prin culoare roșie, fără modificarea scorului.

Eficiența pedagogică a acestui mecanism vine din faptul că reacția sistemului este imediată și ușor de interpretat. Utilizatorii nu trebuie să citească mesaje lungi sau explicații tehnice; culoarea, scorul și mișcarea scenei centrale transmit instantaneu rezultatul. Această abordare este adecvată pentru elevi mai mici sau pentru activități ritmate, în care fluiditatea și claritatea sunt mai importante decât analiza textuală detaliată a fiecărui pas.

3.6. Extensibilitate și posibilități de evoluție

Deși proiectul are dimensiuni reduse, el este conceput într-o formă care permite extindere. Întrebările pot fi multiplicate fără schimbări în cod, funcțiile de actualizare pot fi extrase în module separate, iar interfața poate primi noi componente, precum cronometru, indicator de rundă, istoric al răspunsurilor sau niveluri de dificultate. Aceste direcții ar transforma aplicația dintr-o demonstrație solidă într-un produs educațional mai bogat.

O altă dimensiune a extensibilității este integrarea ulterioară cu servicii externe. În locul unui fișier JSON local, întrebările ar putea fi furnizate de un API. Rezultatele ar putea fi salvate într-o

bază de date, iar profesorul ar putea primi rapoarte sau statistici. Faptul că proiectul actual este clar și bine delimitat face aceste evoluții mai ușor de planificat, deoarece fiecare viitoare funcționalitate poate fi raportată la o bază tehnică deja stabilă.

4. Implementarea interfeței cu HTML și CSS

4.1. Structura HTML a interfeței

Fișierul HTML definește o interfață compusă din trei coloane: două coloane laterale pentru echipe și o coloană centrală pentru informațiile generale și pentru scena vizuală a jocului. Fiecare coloană laterală conține antetul echipei, scorul, zona de întrebare și calculatorul numeric. Coloana din centru reunește titlul aplicației, starea jocului, descrierea provocării curente, scena multimedia și butoanele globale de control. Această distribuție oferă o simetrie clară și susține lectura rapidă a ecranului.

Din punct de vedere al ancorării logice, HTML-ul furnizează identificatori expliciți pentru toate elementele care trebuie actualizate din JavaScript: scoruri, întrebări, afișaje numerice, stare, text central și elementul video. O asemenea abordare simplifică manipularea DOM-ului și face codul ușor de urmărit. Chiar dacă documentul nu folosește etichete semantice mai avansate sau componente reutilizabile, structura existentă este suficient de ordonată pentru dimensiunea proiectului.

4.2. Organizarea vizuală prin CSS

Fișierul CSS realizează trecerea de la o simplă structură de elemente la o interfață recognoscibilă și motivantă. Layout-ul general este construit cu CSS Grid, prin formula `1fr 2fr 1fr`, ceea ce alocă mai mult spațiu zonei centrale fără a diminua lizibilitatea panourilor laterale. Fundalul cu gradient, cardurile albe, umbrele și colțurile rotunjite creează o imagine prietenoasă, apropiată de cea a aplicațiilor moderne și potrivită pentru activități cu componentă ludică.

Stilizarea nu are doar rol estetic, ci și funcțional. Dimensiunile mari ale scorului și ale butoanelor facilitează utilizarea la distanță, culorile diferențiază rapid echipele, iar animațiile discrete de hover sugerează interactivitatea. CSS-ul gestionează și starea de blocare a coloanelor, printr-un strat semi-transparent cu mesaj explicit. Astfel, utilizatorul înțelege imediat dacă poate sau nu să interacționeze cu zona respectivă, fără explicații suplimentare.

4.3. Panourile de interacțiune pentru echipe

Fiecare echipă dispune de un panou propriu în care sunt reunite elementele strict necesare rezolvării sarcinii: întrebarea, scorul și calculatorul numeric. Acest design reduce confuzia și limitează numărul de informații concurente. Elevii nu trebuie să caute comenzile în mai multe zone ale ecranului; tot ce au nevoie pentru acțiunea curentă este grupat local. De asemenea, faptul că

panourile sunt simetrice transmite un mesaj de echitate și elimină diferențele vizuale nejustificate dintre cele două echipe.

Calculatorul virtual este o alegere inspirată pentru mediile touch și pentru activitățile proiectate. Introducerea răspunsului prin butoane mari este mai sigură decât scrierea de la tastatură atunci când există mai mulți participanți sau când operatorul lucrează de la distanță față de ecran. În plus, butoanele dedicate pentru ștergere și trimitere clarifică secvența de lucru: introduci numărul, îl verifici pe afișaj și îl trimiți pentru validare.

4.4. Zona centrală și componenta multimedia

Coloana centrală are rolul de a comunica starea globală a jocului și de a produce impactul vizual principal. Antetul prezintă titlul aplicației și scopul general, iar câmpul de status oferă mesaje scurte despre evoluția jocului. Sub acestea se află zona „Current Challenge”, care poate fi extinsă în versiunile viitoare pentru a afișa informații pedagogice suplimentare. Elementul central rămâne scena video, unde progresul este sugerat prin deplasarea conținutului și prin schimbarea clipului la final.

Utilizarea resurselor media `idle.mp4`, `win-blue-left.mp4` și `win-red-right.mp4` oferă jocului o expresivitate mult mai mare decât ar fi posibil prin simple texte. În special pentru copii sau pentru activități de grup, animația finală funcționează ca recompensă vizuală și închide jocul într-o notă clară. În același timp, această soluție rămâne relativ simplă tehnic, deoarece se bazează pe elementul video nativ din browser și pe schimbarea sursei în funcție de starea jocului.

4.5. Elemente de responsive design și accesibilitate

Interfața a fost proiectată în primul rând pentru ecrane late, ceea ce este firesc pentru un produs folosit în laborator sau la proiector. Cu toate acestea, multe dintre deciziile de design susțin și redimensionarea pe ecrane mai mici: coloanele sunt definite procentual, butoanele sunt suficient de mari, iar blocurile principale sunt independente vizual. Aplicația poate funcționa și pe tablete, deși pentru o utilizare intensă pe telefoane ar fi necesare ajustări suplimentare de layout și de spațiere.

Din punct de vedere al accesibilității, proiectul are câteva puncte forte și câteva limite. Punctele forte sunt contrastul bun în zonele de status, dimensiunile mari ale controalelor și claritatea fluxului de interacțiune. Limitele țin de lipsa unor etichete ARIA mai detaliate, de dependența puternică de culoare pentru feedback și de faptul că interacțiunea este gândită în primul rând pentru click sau touch, nu pentru tastatură. Aceste observații sunt importante deoarece arată direcțiile în care aplicația poate deveni mai incluzivă.

4.6. Evaluarea interfeței și direcții de rafinare

Interfața produsului are meritul de a comunica instantaneu natura jocului: două echipe, un obiectiv comun și o competiție vizibilă. Elementele principale sunt ușor de recunoscut, iar zona centrală creează o experiență memorabilă. Acest lucru este important în context educațional, unde prima impresie și ușurința de utilizare influențează direct disponibilitatea elevilor de a participa. Un joc simplu, dar clar și expresiv, este adesea mai eficient decât o aplicație încărcată cu funcții neesențiale.

Totuși, analiza interfeței evidențiază și posibilități de rafinare. În cod există un banner de câștigător pregătit stilistic, dar neactivat logic, ceea ce sugerează o funcționalitate incomplet exploatată. De asemenea, textul din zona „Current Challenge” poate transmite informații mai specifice despre întrebările curente, iar modul de afișare a erorilor ar putea include și explicații succinte. Aceste ajustări nu schimbă arhitectura de bază, dar ar crește coerența și valoarea pedagogică a produsului.

5. Implementarea logicii aplicației în JavaScript

5.1. Încărcarea întrebărilor și utilizarea funcției fetch

Logica aplicației începe cu încărcarea întrebărilor din fișierul JSON, realizată prin funcția `fetch`. Această alegere demonstrează un model important al dezvoltării web: conținutul poate fi separat de logică și accesat asincron atunci când aplicația pornește. Dacă operația reușește, rezultatul este salvat în variabila `questions`; în caz de eroare, consola primește un mesaj util pentru depanare. Chiar dacă mecanismul este simplu, el introduce utilizatorul și dezvoltatorul în lucrul cu resurse externe și cu promisiuni.

Din perspectivă practică, încărcarea asincronă înseamnă că interfața poate fi pregătită independent de momentul exact în care datele devin disponibile. Pentru proiecte mici acest avantaj poate părea minor, dar el reflectă o bună practică importantă. Mai mult, dacă sursa întrebărilor ar fi mutată ulterior într-un serviciu online, modelul conceptual al aplicației ar rămâne similar. Prin urmare, soluția actuală reprezintă și o bază de învățare pentru extinderi ulterioare.

5.2. Selectarea întrebărilor și evitarea repetării

Funcția `getRandomQuestion` răspunde unei probleme simple, dar importante: cum poate fi oferită o întrebare nouă fiecărei echipe fără repetiții premature. Implementarea folosește câte un `Set` pentru fiecare echipă, în care sunt memorate indicii întrebărilor deja utilizate. Dacă toate întrebările au fost parcurse, setul este golit și selecția poate reîncepe. Această abordare este potrivită pentru o bancă mică de date și poate fi înțeleasă cu ușurință de către un dezvoltator aflat la început.

Este util de observat și o subtilitate tehnică: funcția construiește mai întâi o listă cu întrebări disponibile, dar selectează apoi un indice aleatoriu din lista totală și recurge la apel recursiv atunci când nimerește o întrebare deja folosită. Pentru dimensiunea actuală a setului de date, costul este neglijabil, însă într-o variantă extinsă ar fi mai eficientă selectarea directă din lista filtrată. Analiza acestui detaliu este relevantă deoarece arată că funcționalitatea este corectă, dar poate fi optimizată în viitor.

5.3. Gestionarea intrării numerice și validarea răspunsurilor

Introducerea răspunsului se face prin funcțiile `appendNumber`, `clearDisplay` și `submitAnswer`. Prima actualizează afișajul numeric al echipei active, a doua îl resetează la valoarea implicită, iar a treia compară răspunsul introdus cu valoarea corectă din întrebarea curentă. Din punct de vedere al designului logic, aceste funcții au responsabilități bine delimitate și sunt ușor

de urmărit, ceea ce este un avantaj pentru mentenanță și pentru explicarea codului în cadrul susținerii lucrării.

Validarea se bazează pe compararea a două șiruri de caractere. Soluția este suficientă în contextul actual, deoarece răspunsurile sunt numere întregi scurte și bine controlate. Totuși, în aplicații mai complexe ar fi recomandată convertirea explicită la tip numeric, tratarea spațiilor și validarea mai tolerantă a formatului. Menționarea acestei observații este importantă pentru că arată înțelegerea diferenței dintre o soluție minimă funcțională și o soluție robustă, pregătită pentru date mai variate.

5.4. Actualizarea scorului și feedbackul vizual

Atunci când răspunsul este corect, scorul echipei este incrementat, iar funcțiile `updateScore` și `updateTugVideoPosition` sincronizează interfața cu noua stare. În plus, zona întrebării este colorată temporar în verde, ceea ce oferă confirmare imediată chiar înainte ca utilizatorul să proceseze schimbarea numerică a scorului. În cazul unui răspuns greșit, aceeași zonă este colorată temporar în roșu, iar scorul rămâne neschimbat. Această combinație de feedback numeric și cromatic este ușor de înțeles și eficientă în contexte de joc rapid.

Funcția `updateTugVideoPosition` realizează legătura cea mai vizibilă dintre performanță și scenă. Deplasarea clipului video în funcție de diferența de scor creează senzația că balanța competiției se schimbă în timp real. Chiar dacă implementarea este grafic simplificată, ea produce efectul dorit: răspunsurile nu sunt doar corecte sau greșite, ci au consecințe vizibile în spațiul comun al jocului. Acesta este unul dintre punctele forte ale aplicației, deoarece transformă o activitate cognitivă abstractă într-un eveniment vizual comun pentru întreaga clasă.

5.5. Ciclul de viață al jocului: start, finalizare, resetare

Funcția `startGame` pregătește întreaga aplicație pentru o nouă rundă: resetează scorurile, reactivează coloanele echipelor, reinițializează seturile de întrebări folosite, reîncarcă scena video și distribuie primele întrebări. Această secvență este importantă pentru că stabilește un început controlat și previzibil. Utilizatorii nu ajung într-o stare intermediară ambiguă, iar profesorul poate reporni activitatea de fiecare dată în aceleași condiții.

Finalizarea și resetarea sunt tratate prin funcțiile `endGame` și `resetGame`. Prima oprește jocul și schimbă scena într-un clip final asociat echipei câștigătoare. A doua revine la starea inițială, reactualizează mesajele de interfață și pregătește o nouă sesiune. Împreună, aceste funcții oferă aplicației un ciclu de viață complet și coerent. Pentru utilizare pedagogică, o astfel de claritate este

esențială: profesorul trebuie să poată începe, opri și relua activitatea fără riscul de a păstra accidental stări din runda anterioară.

5.6. Mentenabilitate, claritate și oportunități de refactorizare

Codul JavaScript este compact și lizibil, ceea ce reprezintă un avantaj pentru un proiect educațional. Numele funcțiilor sunt sugestive, iar fluxul principal poate fi urmărit fără instrumente suplimentare. În același timp, analiza detaliată arată câteva direcții de refactorizare utile: separarea utilitarelor de actualizare DOM de logica jocului, introducerea unei funcții unice pentru schimbarea mesajelor de stare și reorganizarea codului în module pentru interfață, date și reguli de joc.

O altă direcție importantă este introducerea testării automate pentru funcțiile pure și pentru regulile de selecție a întrebărilor. Deși produsul actual este suficient de mic pentru a fi verificat manual, trecerea la o bază de întrebări mai mare sau la moduri de joc suplimentare ar face utilă o astfel de disciplină. Prin urmare, lucrarea evidențiază nu doar ceea ce este implementat, ci și pașii necesari pentru evoluția proiectului spre o formă mai robustă și mai scalabilă.

6. Testare, evaluare și perspective de dezvoltare

6.1. Metodologia de testare

Evaluarea aplicației a fost gândită în primul rând ca o testare funcțională orientată pe scenarii de utilizare. Această alegere este firească pentru un produs front-end care rulează local și ale cărui funcții principale sunt ușor de observat în interfață. Testarea a urmărit comportamentul fiecărui flux critic: încărcarea întrebărilor, pornirea jocului, introducerea răspunsurilor, modificarea scorului, finalizarea rundei și revenirea la starea inițială.

Pe lângă verificarea funcțională, a fost realizată și o analiză de cod pentru a identifica zonele în care comportamentul este corect, dar poate fi optimizat. Această dublă abordare este utilă deoarece unele probleme nu apar imediat în interfață, însă devin vizibile la examinarea algoritmilor sau a dependențelor dintre funcții. Într-o lucrare de licență, un astfel de mod de validare demonstrează atât înțelegerea produsului ca experiență de utilizare, cât și capacitatea de analiză tehnică.

6.2. Scenarii funcționale de verificare

Scenariile de test selectate au fost construite astfel încât să acopere toate stările importante ale sistemului. Fiecare scenariu verifică o acțiune principală, un rezultat așteptat și, acolo unde este relevant, o condiție de regresie. O astfel de organizare oferă claritate și permite refolosirea listei de test după fiecare modificare viitoare a codului.

Tabelul următor sintetizează cazurile de test reprezentative pentru aplicația actuală. El poate servi atât ca instrument de verificare tehnică, cât și ca suport pentru demonstrația practică în cadrul susținerii, deoarece evidențiază exact ce trebuie urmărit în timpul rulării produsului.

Cod	Scenariu	Acțiune	Rezultat așteptat
T1	Pornirea jocului din starea inițială	START GAME	Scorurile se resetează, întrebările se încarcă, panourile se deblochează
T2	Răspuns corect pentru o echipă	Introducere rezultat corect și SUBMIT	Scorul crește cu 1 și apare feedback pozitiv
T3	Răspuns greșit	Introducere rezultat incorect și SUBMIT	Scorul rămâne neschimbat și apare feedback negativ
T4	Atingerea pragului de victorie	Repetarea răspunsurilor corecte până la 10 puncte	Jocul se oprește și este afișat clipul de câștig
T5	Resetarea completă după finalizare	RESET	Interfața revine la starea inițială și scorurile devin 0
T6	Evitarea repetării premature	Parcurgerea mai multor întrebări succesive	Întrebările deja utilizate nu se repetă până la epuizarea setului

T7	Blocarea interacțiunii înainte de start	Apăsare pe butoanele echipelor înainte de START	Panourile sunt indisponibile și afișează mesajul de blocare

6.3. Valoarea pedagogică a aplicației

Dincolo de componenta software, proiectul are valoare pedagogică prin felul în care structurează activitatea elevilor. Jocul îi obligă să reacționeze rapid, să colaboreze și să observe relația dintre efort și rezultat. Pentru profesor, aceasta înseamnă o creștere a energiei lecției și o posibilitate în plus de a lucra cu clasa într-un ritm intens, fără a abandona conținutul curricular de bază.

Un alt beneficiu este vizibilitatea progresului. În multe activități tradiționale, elevul primește confirmarea doar prin intervenția profesorului sau după corectarea unui set de exerciții. Aici, răspunsul corect produce imediat un efect observabil de întreaga clasă. Această transparență consolidează motivația și poate susține inclusiv discuții despre strategie, atenție și cooperare. Astfel, aplicația nu este utilă doar pentru exersarea aritmeticii, ci și pentru dinamica grupului și pentru participarea activă.

6.4. Limitări tehnice și funcționale

În varianta actuală, aplicația nu salvează progresul, nu are profiluri de utilizatori și nu include un mecanism prin care profesorul să poată adăuga întrebări din interfață. Aceste limitări sunt firești pentru un proiect front-end compact, dar trebuie consemnate pentru a nu supraevalua produsul. De asemenea, lipsa unui cronometru sau a unor niveluri de dificultate face ca toate rundele să aibă un ritm similar, fără adaptare la nivelul grupei.

O altă limită este dependența de resurse locale și de structura fixă a fișierelor. Dacă fișierul JSON nu se încarcă sau dacă aplicația este deschisă dintr-un context care blochează `fetch`, jocul nu mai poate furniza întrebări. În plus, unele elemente de interfață există doar parțial în cod, cum este bannerul de câștigător nefolosit. Aceste observații nu diminuează utilitatea soluției, dar arată zonele în care produsul trebuie consolidat pentru utilizare repetată pe termen lung.

6.5. Direcții de dezvoltare ulterioară

Cea mai evidentă direcție de dezvoltare este extinderea băncii de întrebări și organizarea ei pe niveluri de dificultate. Aceasta ar permite folosirea aplicației la mai multe clase și în mai multe momente ale lecției. O evoluție naturală ar fi și introducerea unui panou simplu pentru profesor, din care să poată selecta tipul de exerciții, numărul de puncte necesare pentru victorie sau viteza jocului.

La nivel tehnic, proiectul poate fi extins prin adăugarea unui backend ușor care să gestioneze seturi de întrebări, rezultate și statistici. O altă direcție relevantă este accesibilitatea: suport complet pentru tastatură, opțiuni de contrast ridicat, sunete configurabile și texte alternative mai bogate. În acest fel, aplicația ar putea evolua dintr-un joc demonstrativ spre un instrument educațional adaptabil și reutilizabil într-o varietate mai mare de contexte.

6.6. Recomandări pentru exploatarea practică

Pentru utilizarea eficientă a produsului în clasă se recomandă lansarea lui înainte de începerea activității și verificarea încărcării complete a resurselor media și a întrebărilor. În forma actuală, pornirea se poate face local după instalarea dependențelor și rularea scriptului de start definit în `package.json`. Deoarece proiectul folosește `serve`, nu este nevoie de configurări suplimentare de server, ceea ce îl face adecvat pentru demonstrații rapide sau lecții unde timpul de pregătire este limitat.

Din perspectivă didactică, aplicația funcționează cel mai bine atunci când profesorul formulează explicit regulile și păstrează controlul asupra ritmului. Este utilă alternarea rundelor rapide cu momente scurte de comentariu, în care să se discute greșelile tipice sau strategiile de calcul. Astfel, jocul nu rămâne doar o activitate competitivă, ci devine parte integrantă a procesului de predare și consolidare. Recomandarea generală este folosirea lui ca instrument complementar, nu ca substitut complet pentru exercițiul explicat și reflectat.

Concluzii și recomandări

Lucrarea a urmărit proiectarea, descrierea și evaluarea unei aplicații educaționale de tip joc, destinată activităților de grup și construită cu tehnologii web accesibile. Analiza a arătat că soluția propusă răspunde unei nevoi reale din procesul didactic: transformarea exercițiilor repetitive de calcul într-o activitate mai dinamică, mai motivantă și mai vizibilă pentru întreaga clasă. Prin combinarea elementelor de competiție controlată cu feedback vizual imediat, produsul reușește să creeze o experiență coerentă și ușor de înțeles.

Din punct de vedere tehnic, aplicația demonstrează că un proiect compact, alcătuit din HTML, CSS, JavaScript și un fișier JSON, poate oferi o valoare educațională semnificativă atunci când este bine structurat. Modelul de stare `gameState`, separarea clară a conținutului de logică și organizarea interfeței în trei zone distincte reprezintă alegeri de proiectare potrivite pentru obiectivul urmărit. În același timp, lucrarea a evidențiat și limitele curente ale produsului, ceea ce oferă o bază realistă pentru evaluare și dezvoltare.

Un rezultat important al proiectului este demonstrarea faptului că mecanicile de joc nu trebuie să fie complexe pentru a produce implicare. În cazul de față, doar câteva reguli simple, un scor vizibil și o scenă simbolică de tip tragere de funie sunt suficiente pentru a transforma un set de exerciții într-o activitate colectivă atractivă. Această observație are relevanță practică pentru cadrele didactice și pentru dezvoltatorii de instrumente educaționale care caută soluții cu cost tehnic redus și impact imediat.

Din perspectiva viitorului proiectului, direcțiile prioritare sunt extinderea conținutului, creșterea accesibilității, introducerea unui mecanism de administrare și consolidarea robusteții tehnice. Totuși, chiar și în forma actuală, aplicația este aptă pentru demonstrații, exerciții rapide și activități de consolidare. Prin urmare, lucrarea confirmă ipoteza de la care a pornit: un joc educațional simplu, bine proiectat, poate susține învățarea activă și poate oferi un cadru atractiv pentru exersarea matematicii în echipă.

Bibliografie

[1] MDN Web Docs. HTML basics. Disponibil la: https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Structuring_content/Basic_HTML_syntax
[2] MDN Web Docs. CSS first steps. Disponibil la: https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Styling_basics
[3] MDN Web Docs. JavaScript first steps. Disponibil la: https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Scripting
[4] MDN Web Docs. Fetch API. Disponibil la: https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API
[5] javascript.info. The JavaScript language. Disponibil la: https://javascript.info/first-steps
[6] W3Schools. JSON Introduction. Disponibil la: https://www.w3schools.com/js/js_json.asp
[7] web.dev. Responsive web design basics. Disponibil la: https://web.dev/articles/responsive-web-design-basics

Anexe

Anexele includ structura proiectului și listingurile sursă reprezentative pentru a demonstra concret modul în care soluția descrisă în capitolele anterioare este implementată. Aceste materiale tehnice completează explicația teoretică și permit verificarea directă a relației dintre cerințe, proiectare și codul rezultat.

Anexa 1. Structura proiectului și resursele folosite

Element	Descriere
index.html	Definește structura completă a interfeței, zonele vizuale și ancorele DOM pentru actualizare din JavaScript
styles.css	Controlează layout-ul pe trei coloane, stilurile butoanelor, culorile echipelor și efectele vizuale
game.js	Conține logica jocului, gestionarea stării, încărcarea întrebărilor și reacțiile de interfață
questions.json	Stochează banca de întrebări și răspunsuri într-un format simplu și extensibil
assets/	Include imaginea tematică și videoclipurile folosite pentru feedback vizual
package.json	Păstrează scriptul de pornire și dependența folosită pentru servirea locală a aplicației
assets/image.png	Imagine tematică utilizată în documentație și în prezentarea conceptului
assets/idle.mp4 / win-*.mp4	Clipuri video folosite pentru scena centrală și finalizarea jocului

Anexa 2. Listing complet index.html

```
001 | <!DOCTYPE html>
002 | <html lang="en">
003 | <head>
004 |     <meta charset="UTF-8">
005 |     <meta name="viewport" content="width=device-width, initial-scale=1.0">
006 |     <title>Teams Quiz Game</title>
007 |     <link rel="stylesheet" href="styles.css">
008 | </head>
009 | <body>
010 |     <div class="container">
011 |         <!-- Left Column - Team 1 -->
012 |         <div class="column team-column blocked">
013 |             <div class="team-header team1-header">TEAM 1</div>
014 |             <div class="score" id="score1">0</div>
015 |             <div class="question-display" id="question1">Waiting for game to start...</div>
016 |             <div class="calculator">
017 |                 <div class="calc-display" id="display1">0</div>
018 |                 <button class="calc-btn" onclick="appendNumber(1, '7')">7</button>
019 |                 <button class="calc-btn" onclick="appendNumber(1, '8')">8</button>
020 |                 <button class="calc-btn" onclick="appendNumber(1, '9')">9</button>
021 |                 <button class="calc-btn" onclick="appendNumber(1, '4')">4</button>
022 |                 <button class="calc-btn" onclick="appendNumber(1, '5')">5</button>
023 |                 <button class="calc-btn" onclick="appendNumber(1, '6')">6</button>
024 |                 <button class="calc-btn" onclick="appendNumber(1, '1')">1</button>
025 |                 <button class="calc-btn" onclick="appendNumber(1, '2')">2</button>
026 |                 <button class="calc-btn" onclick="appendNumber(1, '3')">3</button>
027 |                 <button class="calc-btn zero" onclick="appendNumber(1, '0')">0</button>
028 |                 <button class="calc-btn clear" onclick="clearDisplay(1)">C</button>
029 |                 <button class="calc-btn submit" onclick="submitAnswer(1)">SUBMIT</button>
030 |             </div>
031 |         </div>
032 |
033 |         <!-- Middle Column - Animation -->
034 |         <div class="column middle-column">
035 |             <div class="game-header">
036 |                 <h1>🏆 Teams Quiz Game 🏆</h1>
037 |                 <p>Race to 10 correct answers!</p>
038 |             </div>
039 |
040 |             <div class="status" id="status">Press START to begin the game</div>
041 |
042 |             <div class="current-question">
043 |                 <h2>Current Challenge</h2>
044 |                 <p id="currentQuestionText">No active question</p>
045 |             </div>
046 |
047 |             <div class="race-track">
048 |                 <div class="tug-stage">
049 |                     <video
050 |                         class="tug-video"
051 |                         id="tugVideo"
052 |                         src="assets/idle.mp4"
053 |                         autoplay
054 |                         muted
055 |                         loop
056 |                         playsinline
057 |                     ></video>
058 |                     <div class="tug-center-line" aria-hidden="true"></div>
059 |                 </div>
060 |             </div>
061 |
062 |             <div class="controls">
063 |                 <button class="btn btn-start" onclick="startGame()">START GAME</button>
064 |                 <button class="btn btn-reset" onclick="resetGame()">RESET</button>
065 |             </div>
```

```
066 |         </div>
067 |
068 |         <!-- Right Column - Team 2 -->
069 |         <div class="column team-column blocked">
070 |             <div class="team-header team2-header">TEAM 2</div>
071 |             <div class="score" id="score2">0</div>
072 |             <div class="question-display" id="question2">Waiting for game to start...</div>
073 |             <div class="calculator">
074 |                 <div class="calc-display" id="display2">0</div>
075 |                 <button class="calc-btn" onclick="appendNumber(2, '7')">7</button>
076 |                 <button class="calc-btn" onclick="appendNumber(2, '8')">8</button>
077 |                 <button class="calc-btn" onclick="appendNumber(2, '9')">9</button>
078 |                 <button class="calc-btn" onclick="appendNumber(2, '4')">4</button>
079 |                 <button class="calc-btn" onclick="appendNumber(2, '5')">5</button>
080 |                 <button class="calc-btn" onclick="appendNumber(2, '6')">6</button>
081 |                 <button class="calc-btn" onclick="appendNumber(2, '1')">1</button>
082 |                 <button class="calc-btn" onclick="appendNumber(2, '2')">2</button>
083 |                 <button class="calc-btn" onclick="appendNumber(2, '3')">3</button>
084 |                 <button class="calc-btn zero" onclick="appendNumber(2, '0')">0</button>
085 |                 <button class="calc-btn clear" onclick="clearDisplay(2)">C</button>
086 |                 <button class="calc-btn submit" onclick="submitAnswer(2)">SUBMIT</button>
087 |             </div>
088 |         </div>
089 |     </div>
090 |
091 |     <div class="winner-banner" id="winnerBanner">
092 |         <h2 id="winnerText">🎉 WINNER! 🎉</h2>
093 |     </div>
094 |
095 |     <script src="game.js"></script>
096 | </body>
097 | </html>
```

Anexa 3. Listing complet game.js

```
001 | // Question bank - loaded from external file
002 | let questions = [];
003 |
004 | // Load questions from JSON file
005 | fetch('questions.json')
006 |     .then(response => response.json())
007 |     .then(data => {
008 |         questions = data;
009 |         console.log('Questions loaded successfully:', questions.length);
010 |     })
011 |     .catch(error => {
012 |         console.error('Error loading questions:', error);
013 |     });
014 |
015 | let gameState = {
016 |     active: false,
017 |     scores: { 1: 0, 2: 0 },
018 |     currentQuestions: { 1: null, 2: null },
019 |     usedQuestions: { 1: new Set(), 2: new Set() },
020 |     displays: { 1: "0", 2: "0" }
021 | };
022 |
023 | function setTeamColumnsBlocked(blocked) {
024 |     document.querySelectorAll('.team-column').forEach((column) => {
025 |         column.classList.toggle('blocked', blocked);
026 |     });
027 | }
028 |
029 | function setTugCenterLineHidden(hidden) {
030 |     const centerLine = document.querySelector('.tug-center-line');
031 |     if (!centerLine) return;
032 |     centerLine.style.display = hidden ? 'none' : 'block';
033 | }
034 |
035 | function getRandomQuestion(team) {
036 |     const available = questions.filter((_, idx) => !gameState.usedQuestions[team].has(idx));
037 |     if (available.length === 0) {
038 |         gameState.usedQuestions[team].clear();
039 |         return getRandomQuestion(team);
040 |     }
041 |     const randomIdx = Math.floor(Math.random() * questions.length);
042 |     if (!gameState.usedQuestions[team].has(randomIdx)) {
043 |         gameState.usedQuestions[team].add(randomIdx);
044 |         return questions[randomIdx];
045 |     }
046 |     return getRandomQuestion(team);
047 | }
048 |
049 | function startGame() {
050 |     if (gameState.active) return;
051 |
052 |     gameState.active = true;
053 |     setTeamColumnsBlocked(false);
054 |     setTugCenterLineHidden(false);
055 |     gameState.scores = { 1: 0, 2: 0 };
056 |     gameState.usedQuestions = { 1: new Set(), 2: new Set() };
057 |
058 |     const tugVideo = document.getElementById('tugVideo');
059 |     if (tugVideo) {
060 |         tugVideo.src = 'assets/idle.mp4';
061 |         tugVideo.loop = true;
062 |         tugVideo.currentTime = 0;
063 |         tugVideo.play().catch(() => {});
064 |     }
065 |
```

```
066 |     updateScore(1);
067 |     updateScore(2);
068 |     updateTugVideoPosition();
069 |
070 |     loadNewQuestion(1);
071 |     loadNewQuestion(2);
072 |
073 |     document.getElementById('status').textContent = 'Game in progress - First to 10 wins!';
074 |     document.getElementById('currentQuestionText').textContent = 'Both teams have their
   questions!';
075 | }
076 |
077 | function loadNewQuestion(team) {
078 |     const question = getRandomQuestion(team);
079 |     gameState.currentQuestions[team] = question;
080 |     document.getElementById(`question${team}`).textContent = question.question;
081 | }
082 |
083 | function appendNumber(team, num) {
084 |     if (!gameState.active) return;
085 |
086 |     if (gameState.displays[team] === "0") {
087 |         gameState.displays[team] = num;
088 |     } else {
089 |         gameState.displays[team] += num;
090 |     }
091 |     document.getElementById(`display${team}`).textContent = gameState.displays[team];
092 | }
093 |
094 | function clearDisplay(team) {
095 |     gameState.displays[team] = "0";
096 |     document.getElementById(`display${team}`).textContent = "0";
097 | }
098 |
099 | function submitAnswer(team) {
100 |     if (!gameState.active) return;
101 |
102 |     const userAnswer = gameState.displays[team];
103 |     const correctAnswer = gameState.currentQuestions[team].answer;
104 |
105 |     if (userAnswer === correctAnswer) {
106 |         gameState.scores[team]++;
107 |         updateScore(team);
108 |         updateTugVideoPosition();
109 |
110 |         // Flash the question display
111 |         const questionDisplay = document.getElementById(`question${team}`);
112 |         questionDisplay.style.background = '#27ae60';
113 |         questionDisplay.style.color = 'white';
114 |         setTimeout(() => {
115 |             questionDisplay.style.background = '#f8f9fa';
116 |             questionDisplay.style.color = '#333';
117 |         }, 500);
118 |
119 |         if (gameState.scores[team] >= 10) {
120 |             endGame(team);
121 |             return;
122 |         }
123 |
124 |         loadNewQuestion(team);
125 |         clearDisplay(team);
126 |
127 |         document.getElementById('status').textContent = `Team ${team} scored! 🎯`;
128 |     } else {
129 |         // Flash red for wrong answer
130 |         const questionDisplay = document.getElementById(`question${team}`);
131 |         questionDisplay.style.background = '#e74c3c';
```

```
132 |         questionDisplay.style.color = 'white';
133 |         setTimeout(() => {
134 |             questionDisplay.style.background = '#f8f9fa';
135 |             questionDisplay.style.color = '#333';
136 |         }, 500);
137 |     }
138 | }
139 |
140 | function updateScore(team) {
141 |     document.getElementById(`score${team}`).textContent = gameState.scores[team];
142 | }
143 |
144 | function updateTugLinePosition() {
145 |     const centerLine = document.querySelector('.tug-center-line');
146 |     if (!centerLine) return;
147 |
148 |     const scoreDelta = gameState.scores[1] - gameState.scores[2];
149 |     const maxOffset = 180;
150 |     const offsetPerPoint = 18;
151 |     const offset = Math.max(-maxOffset, Math.min(maxOffset, scoreDelta * offsetPerPoint));
152 |
153 |     centerLine.style.transform = `translateX(calc(-50% + ${offset}px))`;
154 | }
155 |
156 | function updateTugVideoPosition() {
157 |     const tugVideo = document.getElementById('tugVideo');
158 |     if (!tugVideo) return;
159 |
160 |     const scoreDelta = gameState.scores[2] - gameState.scores[1];
161 |     const maxOffset = 180;
162 |     const offsetPerPoint = 18;
163 |     const offset = Math.max(-maxOffset, Math.min(maxOffset, scoreDelta * offsetPerPoint));
164 |
165 |     tugVideo.style.transform = `translateX(${offset}px)`;
166 | }
167 |
168 | function endGame(winner) {
169 |     gameState.active = false;
170 |     setTeamColumnsBlocked(true);
171 |     setTugCenterLineHidden(true);
172 |     document.getElementById('status').textContent = `🏆 Team ${winner} WINS! 🏆`;
173 |
174 |     const tugVideo = document.getElementById('tugVideo');
175 |     if (tugVideo) {
176 |         tugVideo.style.transform = 'translateX(0px)';
177 |         tugVideo.src = winner === 1 ? 'assets/win-blue-left.mp4' :
    'assets/win-red-right.mp4';
178 |         tugVideo.loop = false;
179 |         tugVideo.currentTime = 0;
180 |         tugVideo.play().catch(() => {});
181 |     }
182 | }
183 |
184 | function resetGame() {
185 |     gameState = {
186 |         active: false,
187 |         scores: { 1: 0, 2: 0 },
188 |         currentQuestions: { 1: null, 2: null },
189 |         usedQuestions: { 1: new Set(), 2: new Set() },
190 |         displays: { 1: "0", 2: "0" }
191 |     };
192 |
193 |     setTeamColumnsBlocked(true);
194 |     setTugCenterLineHidden(false);
195 |
196 |     updateScore(1);
197 |     updateScore(2);
```

```
198 |     updateTugVideoPosition();
199 |     clearDisplay(1);
200 |     clearDisplay(2);
201 |
202 |     document.getElementById('question1').textContent = 'Waiting for game to start...';
203 |     document.getElementById('question2').textContent = 'Waiting for game to start...';
204 |     document.getElementById('status').textContent = 'Press START to begin the game';
205 |     document.getElementById('currentQuestionText').textContent = 'No active question';
206 |
207 |     const tugVideo = document.getElementById('tugVideo');
208 |     if (tugVideo) {
209 |         tugVideo.src = 'assets/idle.mp4';
210 |         tugVideo.loop = true;
211 |         tugVideo.style.transform = 'translateX(0px)';
212 |         tugVideo.currentTime = 0;
213 |         tugVideo.play().catch(() => {});
214 |     }
215 | }
```

Anexa 4. Listing complet styles.css

```
001 | * {
002 |     margin: 0;
003 |     padding: 0;
004 |     box-sizing: border-box;
005 | }
006 |
007 | body {
008 |     font-family: 'Arial', sans-serif;
009 |     background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
010 |     min-height: 100vh;
011 |     padding: 20px;
012 | }
013 |
014 | .container {
015 |     display: grid;
016 |     grid-template-columns: 1fr 2fr 1fr;
017 |     gap: 20px;
018 |     max-width: 1400px;
019 |     margin: 0 auto;
020 |     height: calc(100vh - 40px);
021 | }
022 |
023 | .column {
024 |     background: white;
025 |     border-radius: 15px;
026 |     padding: 20px;
027 |     box-shadow: 0 10px 30px rgba(0, 0, 0, 0.3);
028 | }
029 |
030 | .team-column {
031 |     display: flex;
032 |     flex-direction: column;
033 |     position: relative;
034 | }
035 |
036 | .team-column.blocked {
037 |     pointer-events: none;
038 | }
039 |
040 | .team-column.blocked::after {
041 |     content: 'START GAME TO UNLOCK';
042 |     position: absolute;
043 |     inset: 0;
044 |     display: flex;
045 |     align-items: center;
046 |     justify-content: center;
047 |     text-align: center;
048 |     padding: 24px;
049 |     border-radius: 15px;
050 |     background: rgba(24, 28, 48, 0.62);
051 |     color: #ffffff;
052 |     font-weight: 700;
053 |     letter-spacing: 1px;
054 |     z-index: 10;
055 | }
056 |
057 | .team-header {
058 |     text-align: center;
059 |     padding: 15px;
060 |     border-radius: 10px;
061 |     margin-bottom: 20px;
062 |     color: white;
063 |     font-size: 24px;
064 |     font-weight: bold;
065 | }
```

```
066 |
067 | .team1-header {
068 |     background: linear-gradient(135deg, #4facfe 0%, #00f2fe 100%);
069 | }
070 |
071 | .team2-header {
072 |     background: linear-gradient(135deg, #f093fb 0%, #f5576c 100%);
073 | }
074 |
075 | .score {
076 |     font-size: 48px;
077 |     text-align: center;
078 |     margin: 20px 0;
079 |     color: #333;
080 |     font-weight: bold;
081 | }
082 |
083 | .question-display {
084 |     background: #f8f9fa;
085 |     padding: 20px;
086 |     border-radius: 10px;
087 |     margin-bottom: 20px;
088 |     min-height: 120px;
089 |     display: flex;
090 |     align-items: center;
091 |     justify-content: center;
092 |     text-align: center;
093 |     font-size: 18px;
094 |     color: #333;
095 |     font-weight: 500;
096 | }
097 |
098 | .calculator {
099 |     display: grid;
100 |     grid-template-columns: repeat(3, 1fr);
101 |     gap: 10px;
102 |     margin-bottom: 20px;
103 | }
104 |
105 | .calc-display {
106 |     grid-column: 1 / -1;
107 |     background: #2c3e50;
108 |     color: white;
109 |     padding: 20px;
110 |     border-radius: 10px;
111 |     text-align: right;
112 |     font-size: 32px;
113 |     font-weight: bold;
114 |     min-height: 70px;
115 |     display: flex;
116 |     align-items: center;
117 |     justify-content: flex-end;
118 |     word-wrap: break-word;
119 | }
120 |
121 | .calc-btn {
122 |     padding: 20px;
123 |     font-size: 24px;
124 |     border: none;
125 |     border-radius: 10px;
126 |     cursor: pointer;
127 |     font-weight: bold;
128 |     transition: all 0.2s;
129 |     background: #ecf0f1;
130 |     color: #2c3e50;
131 | }
132 |
```

```
133 | .calc-btn:hover {
134 |     transform: scale(1.05);
135 |     box-shadow: 0 5px 15px rgba(0, 0, 0, 0.2);
136 | }
137 |
138 | .calc-btn:active {
139 |     transform: scale(0.95);
140 | }
141 |
142 | .calc-btn.zero {
143 |     grid-column: 1 / 3;
144 | }
145 |
146 | .calc-btn.clear {
147 |     background: #e74c3c;
148 |     color: white;
149 | }
150 |
151 | .calc-btn.submit {
152 |     background: #27ae60;
153 |     color: white;
154 |     grid-column: 1 / -1;
155 |     font-size: 20px;
156 | }
157 |
158 | /* Middle Column - Animation */
159 | .middle-column {
160 |     display: flex;
161 |     flex-direction: column;
162 |     background: linear-gradient(180deg, #fff 0%, #f8f9fa 100%);
163 | }
164 |
165 | .game-header {
166 |     text-align: center;
167 |     padding: 20px;
168 |     background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
169 |     color: white;
170 |     border-radius: 10px;
171 |     margin-bottom: 20px;
172 | }
173 |
174 | .game-header h1 {
175 |     font-size: 32px;
176 |     margin-bottom: 10px;
177 | }
178 |
179 | .current-question {
180 |     background: white;
181 |     padding: 30px;
182 |     border-radius: 10px;
183 |     margin-bottom: 20px;
184 |     box-shadow: 0 5px 15px rgba(0, 0, 0, 0.1);
185 |     text-align: center;
186 | }
187 |
188 | .current-question h2 {
189 |     color: #667eea;
190 |     margin-bottom: 15px;
191 |     font-size: 24px;
192 | }
193 |
194 | .current-question p {
195 |     font-size: 28px;
196 |     color: #333;
197 |     font-weight: 500;
198 | }
199 |
```

```
.race-track {
    flex: 1;
    position: relative;
    border-radius: 10px;
    padding: 20px;
    margin-bottom: 20px;
    display: flex;
    align-items: center;
    justify-content: center;
    overflow: hidden;
}

.tug-stage {
    position: relative;
    width: min(100%, 720px);
    aspect-ratio: 16 / 9;
    border-radius: 12px;
    overflow: hidden;
    background: #ffffff;
    box-shadow: inset 0 0 0 2px rgba(255, 255, 255, 0.25);
}

.tug-video {
    width: 100%;
    height: 100%;
    object-fit: cover;
    transform: translateX(0);
    transition: transform 0.35s ease-out;
}

.tug-center-line {
    position: absolute;
    top: 0;
    bottom: 0;
    left: 50%;
    transform: translateX(-50%);
    transition: transform 0.35s ease-out;
    border-left: 5px dotted rgba(0, 0, 0, 0.95);
    pointer-events: none;
    filter: drop-shadow(0 0 6px rgba(255, 255, 255, 0.45)) drop-shadow(0 0 8px rgba(0, 0, 0, 0.85));
}

.controls {
    display: flex;
    gap: 10px;
    justify-content: center;
}

.btn {
    padding: 15px 30px;
    font-size: 18px;
    border: none;
    border-radius: 10px;
    cursor: pointer;
    font-weight: bold;
    transition: all 0.2s;
}

.btn-start {
    background: #27ae60;
    color: white;
}

.btn-reset {
    background: #e74c3c;
    color: white;
```

```
266 | }
267 |
268 | .btn:hover {
269 |     transform: scale(1.05);
270 |     box-shadow: 0 5px 15px rgba(0, 0, 0, 0.2);
271 | }
272 |
273 | .status {
274 |     text-align: center;
275 |     padding: 15px;
276 |     background: #fff;
277 |     border-radius: 10px;
278 |     margin-bottom: 10px;
279 |     font-size: 18px;
280 |     font-weight: 500;
281 |     color: #667eea;
282 | }
283 |
284 | .winner-banner {
285 |     position: fixed;
286 |     top: 50%;
287 |     left: 50%;
288 |     transform: translate(-50%, -50%) scale(0);
289 |     background: linear-gradient(135deg, #ffd700 0%, #ffed4e 50%, #ffd700 100%);
290 |     padding: 60px 80px;
291 |     border-radius: 30px;
292 |     box-shadow: 0 20px 60px rgba(0, 0, 0, 0.5);
293 |     text-align: center;
294 |     z-index: 1000;
295 |     transition: transform 0.5s cubic-bezier(0.68, -0.55, 0.265, 1.55);
296 |     border: 5px solid #fff;
297 | }
298 |
299 | .winner-banner h2 {
300 |     font-size: 72px;
301 |     margin-bottom: 20px;
302 |     background: linear-gradient(45deg, #ff0080, #ff8c00, #40e0d0, #ff0080);
303 |     background-size: 200% 200%;
304 |     -webkit-background-clip: text;
305 |     -webkit-text-fill-color: transparent;
306 |     background-clip: text;
307 |     animation: rainbow-text 2s ease infinite;
308 |     text-shadow: 2px 2px 4px rgba(0,0,0,0.2);
309 |     font-weight: bold;
310 | }
311 |
312 | @keyframes rainbow-text {
313 |     0%, 100% {
314 |         background-position: 0% 50%;
315 |     }
316 |     50% {
317 |         background-position: 100% 50%;
318 |     }
319 | }
320 |
321 | @media (max-width: 1024px) {
322 |     .container {
323 |         grid-template-columns: 1fr;
324 |         grid-template-rows: auto auto auto;
325 |     }
326 | }
```

Anexa 5. Listing complet questions.json

```
001 | [
002 |     { "question": "What is 25 + 25?", "answer": "50" },
003 |     { "question": "What is 85 + 13?", "answer": "98" },
004 |     { "question": "What is 42 - 3?", "answer": "39" },
005 |     { "question": "What is 33 - 12?", "answer": "21" },
006 |     { "question": "What is 78 + 13?", "answer": "91" },
007 |     { "question": "What is 67 + 23", "answer": "90" },
008 |     { "question": "What is 49 - 20?", "answer": "29" },
009 |     { "question": "What is 57- 33?", "answer": "24" },
010 |     { "question": "What is 19 + 19?", "answer": "38" },
011 |     { "question": "What is 75 - 18?", "answer": "57" }
012 | ]
```

Anexa 6. Listing complet package.json

```
001 | {
002 |     "name": "teza-buimestru-silvia",
003 |     "version": "1.0.0",
004 |     "description": "Educational game-based application for group activities.",
005 |     "main": "game.js",
006 |     "scripts": {
007 |       "start": "serve"
008 |     },
009 |     "author": "Buimestru Silvia",
010 |     "license": "NO",
011 |     "dependencies": {
012 |       "serve": "^14.2.5"
013 |     }
014 |   }
```